



- Introduzione
 - esame finale:
- Concetti fondamentali
 - qualità del software
 - Processi software
 - Metodi e strumenti
- Attività fondamentali dei processi software
 - STUDIO DI FATTIBILITÀ:
 - CODE AND FIX
 - Acquisizione, analisi e specifica dei requisiti
 - Progettazione e Sviluppo
 - Verifica e Validazione
 - Test dei componenti
 - Test del sistema
 - Test del cliente
 - Test iterativo
 - Evoluzione
- Modelli di processi software



Introduzione

nota:

appunti tratti dalle lezioni e slides del corso di Ingegneria del Software, tenuto dal prof. [Riccio Vincenzo](#)

si inizia di solito alle 8:45

l'anno scorso tutti sono passati tra i vari appelli
materiale didattico su elearning - corso 725

ricevicimento:

- lunedì 15:00-17:00 - L2-02-BC (A2 129)
- volendo anche su teams

esercitazioni in gruppo (2-3 persone), identici a quelli del esame, si faranno a lezione e poi avviene un confronto alla fine della lezione

libro: Ingegneria del software, Ian Sommerville, prima edizione



esame finale:

- scritto carta e penna (120 minuti)
- 3 parti:
 - esercizi su analisi e modellazione
 - esercizi di testing
 - domande di teoria

tutte e 3 le parti devono risultare sufficienti per superare l'esame
orale solo per la lode



Concetti fondamentali

cosa è software?

- è un termine che indica l'insieme dei programmi per computer e la relativa **documentazione** (anche i documenti legali)

noi terremo conto anche di tutta la documentazione prodotta durante lo sviluppo del software, come modelli di progetto, manuali utente, siti web di supporto, ecc.

definizione IEEE:

"software è l'insieme di programmi, procedure, regole e documentazione relativa al funzionamento di un sistema di elaborazione dati"

"software is eating the world" - Marc Andreessen, 2011

si sta diffondendo in tutti i settori, non solo in informatica con una capitalizzazione sempre più grande (ha superato il 1468 miliardi)

con gli LLM non è cambiato il concetto, solo la velocità in cui il software viene sviluppato. Che ci piaccia o no, LLM rimangono software e "programmare" con essi cambia solo il livello di linguaggio usato per lo sviluppo (linguaggio naturale).

Software 1.0:

vedi il problema, pianifichi un algoritmo, progetti un algoritmo e implementi l'algoritmo

Software 2.0:

prepari i dati, definisci un modello, addestri il modello e valuti il modello

Software 3.0:

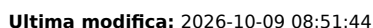
prepara i dati, definisci un modello, addestri il modello, valuti il modello - iterativamente (con un transformer)

software generico:

- sw prodotto autonomamente da una organizzazione per incontrare le necessità di svariati clienti
- il produttore ha controllo sulle specifiche del sw

software custom:

- sw sviluppato da una organizzazione su commissione di uno specifico cliente
- il produttore deve attenersi alle specifiche indicate dal cliente





Le caratteristiche qualitative di un software professionale sono:

- **Accettabilità**
- **Fidatezza e Protezione**
- **Efficienza**
- **Mantenibilità**

L'insieme specifico dei caratteri qualitativi di un sistema software dipende dalla sua applicazione

Sviluppare sw sta diventando sempre più costoso, perchè le operazioni che un sw deve compiere sono sempre più complesse (siamo passati da giocare a pong a creare macchine a guida autonoma)

questo fenomeno venne cercato di risolvere nel 1968 alla NATO Software Engineering Conference di Garmisch, dove il problema venne definito come "**software crisis**"

La soluzione: **ingegneria del software**, che poi fece nascere la **IEEE** (Institute of Electrical and Electronics Engineers), ovvero applicare i principi dell'ingegneria ai processi di sviluppo del software

"L'Ingegneria del Software
è una disciplina
ingegneristica che
riguarda tutti gli aspetti
della produzione del software"

L'Ingegneria del Software (IS) è una disciplina ingegneristica (è l'intersezione tra ingegneria e informatica)



Processi software

Processo software: insieme di attività che porta alla creazione o all'evoluzione di un prodotto software

- Acquisizione, analisi e specifica dei requisiti
- Progettazione e Sviluppo
- Verifica e Validazione
- Evoluzione

Metodi e strumenti

Metodi:

- Approcci strutturati per sviluppare software di qualità, a costi contenuti ed entro i tempi di consegna
- Tali metodi forniscono una guida alle attività dei processi e alla relativa organizzazione

Non sono i comandamenti, ma solide linee guida sviluppate nei corsi di decenni

strumenti:

- Sistemi software usati per aiutare le attività dei processi software (ad es., analisi, modellazione, debugging, testing)

ALCUNE SFIDE PER I PROCESSI SOFTWARE:

- Diversità
- Consegna
- Fiducia
 - creare documentazione, certificazioni, ecc. per far sì che l'utente (o un altro programmatore) possa fidarsi del software
 - ci sono enti standard che certificano la qualità del software
- Scala
 - avere un sw che può essere scalato facilmente qualora il numero di utenti aumenti

PRINCIPI FONDAMENTALI:

- I metodi specifici, le tecniche e gli strumenti utilizzati dipendono dall'organizzazione che sviluppa il software, dal tipo di software e dalle persone coinvolte nel processo di sviluppo
- Non ci sono metodi di ingegneria del software universali che sono applicabili a tutti i sistemi e a tutte le aziende



- I concetti fondamentali dell'ingegneria del software sono **indipendenti dal linguaggio di programmazione** utilizzato per sviluppare il software

Il **processo** va sempre chiarito sin da subito, soprattutto quando si lavora in gruppo, altrimenti l'organizzazione e l'efficienza del gruppo collassa, si devono definire sin da subito i ruoli, le responsabilità e le attività di ogni membro del gruppo, altrimenti il progetto fallisce

FIDATEZZA E PRESTAZIONI:

il sw deve fare quello che deve fare, in maniera efficiente (e senza spreco di risorse), e in modo sicuro

REQUISITI:

- È importante capire e gestire la specifica e i requisiti del software
- Occorre sapere che cosa si aspettano clienti e utenti del sistema, e occorre gestire le loro aspettative in modo che possa essere rilasciato un sistema utile entro i costi e i tempi previsti

RIUSO:

Riutilizzare vecchio codice è spesso più rapido ed economico di scrivere nuovo codice da 0, ma bisogna stare attenti che il codice riutilizzato non abbia bug o sia effettivamente adatto al nuovo contesto.

MODELLO DI PROCESSO:

- Ciascuna organizzazione utilizza il proprio processo
- Il modello di processo software è una rappresentazione semplificata e astratta che descrive l'intero ciclo di vita del software

"come Platone e il mondo delle idee"



Attività fondamentali dei processi software

- Ciascun processo è composto da attività fondamentali
- Tutti i processi software condividono queste attività fondamentali
- Tali attività possono essere organizzate e realizzate in modi diversi in processi diversi

4 ATTIVITÀ FONDAMENTALI DEI PROCESSI SOFTWARE:

- Acquisizione, analisi e specifica dei requisiti
- Progettazione e Sviluppo
- Verifica e Validazione
- Evoluzione



STUDIO DI FATTIBILITÀ:

Stabilisce se lo sviluppo debba essere avviato, ossia:

- Se esiste un mercato per il software
- Se il software sia tecnicamente ed economicamente realistico

Definisce quali sono le alternative possibili e le scelte più ragionevoli, stimando le risorse (finanziarie e umane) necessarie per ciascuna alternativa

se in un progetto ci sono elementi già sviluppati da altri (se open source), ci permette di risparmiare tempo e denaro, se invece l'intero progetto è già stato sviluppato ad un prezzo minore di quello che potremmo proporre, spesso è il caso di abbandonare il progetto.

Fornisce un report di fattibilità con:

- Definizione del problema
- Valutazione Costi/Benefici
- Risorse finanziarie e umane
- Soluzioni alternative
- Tempi di consegna e modalità di sviluppo

Tale studio dovrebbe essere **relativamente rapido** e **poco costoso** (se sto un mese a fare uno studio sui telefoni cellulari per capire quale acquistare e nel mentre esce una nuova generazione, ho buttato il mio tempo perchè devo ricominciare da capo)



CODE AND FIX

Approccio alla produzione di software che consiste nello:

- Scrivere codice
- Aggiustare il codice per correggere errori, migliorare e/o aggiungere funzionalità

correggere errori (anche usando LLMs per fare il debugging), uno ad uno, non è l'approccio ottimale per progetti di grande dimensioni, motivo per cui si necessita di approcci più organizzati allo sviluppo del software.



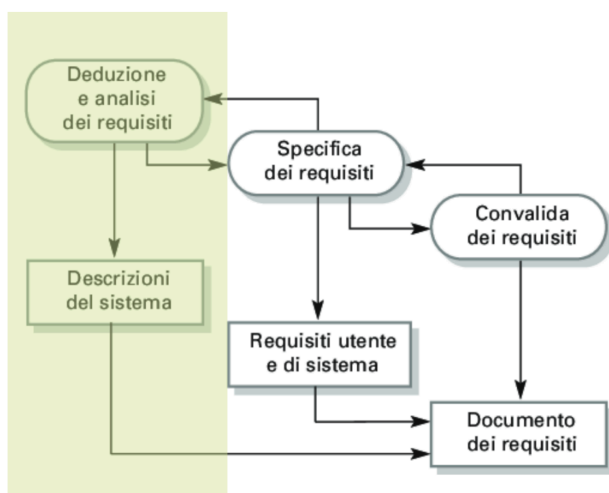
Acquisizione, analisi e specifica dei requisiti

- Attività per stabilire **cosa** il software dovrà fare
- Specificare funzionalità e qualità
- Definisce tramite l'interazione con il committente funzioni, vincoli, prestazioni, interfacce...
- Attività critica

bisogna dedurre e analizzare i requisiti del sw, non solo da parte del cliente, ma anche da parte dei **portatori di interesse** (stakeholder):

- clienti
- committenti
- enti amministrativi esterni

bisogna avere un documento formale con cui si descrive in dettaglio quali sono le funzionalità e i vincoli nel progetto (Che poi verranno convalidati da diversi enti)



La **deduzione** richiede spirito critico e può coinvolgere:

- osservazione di sistemi già esistenti
- discussione con i possibili utenti

La **specifica** traduce le informazioni dedotte in un insieme di requisiti

- Requisiti di **sistema**
- Requisiti **utente**

La **convalida** controlla che i requisiti siano

realistici, coerenti e completi

Durante questo possono essere rilevati errori nel documento dei requisiti e in tal caso il documento dei requisiti deve essere modificato e corretto

Al termine di questo processo, se il progetto rispecchia gli standard, viene approvato.

Tutte queste attività sono intrecciate, perché in ogni fase si possono notare dei difetti o errori avvenuti in una fase precedente



Progettazione e Sviluppo

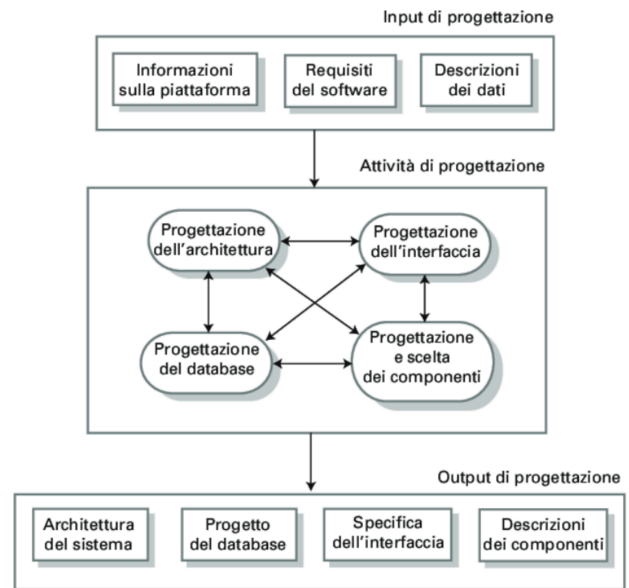
Divisa in due fasi:

- **progettazione**
- **sviluppo**

Le attività della fase di progettazione sono intrecciate e interdipendenti

I progettisti non ottengono immediatamente un risultato completo

Le nuove informazioni sul progetto influenzano le precedenti scelte progettuali



progettazione dell'architettura:

- definire classi, strutture, ecc.

progettare una interfaccia:

- non deve essere necessariamente una interfaccia grafica, ma deve essere in grado di soddisfare tutte le interazioni necessarie con l'esterno
- una volta definita una interfaccia, posso espandere il sw perchè è già suddiviso in moduli

Scelta dei componenti:

- avere componenti riutilizzabili e adattarli
- progettare componenti riutilizzabili
- bisogna mantenere i componenti flessibili, in modo che anche un altro sviluppatore possa riutilizzarli in un contesto diverso

L'output finale di queste fasi è un progetto sw (formale o informale) che descrive:

- La struttura del software che si deve implementare
- I modelli e le strutture di dati usati dal sistema
- Le interfacce tra i componenti del sistema

la parte di **sviluppo** è la meno importante in questo corso. perchè è fortemente dipendente dalle restrizioni o indicazioni di una determinata azienda, cliente, ecc...



Verifica e Validazione

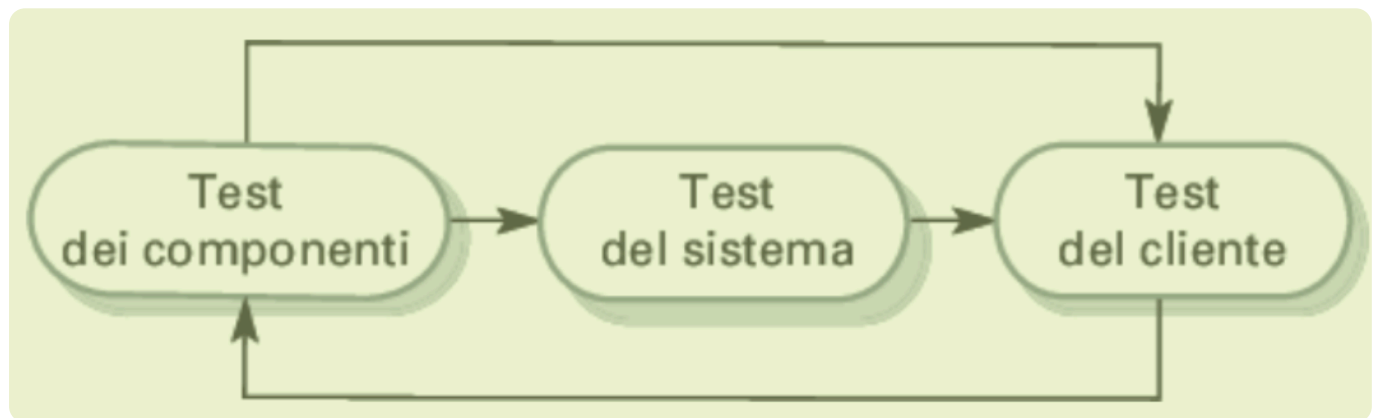
verifica: mostrare che un sistema è conforme alle sue specifiche

validazione: mostrare che un sistema soddisfa le esigenze del cliente

queste due fasi sono complementari tra di loro, perchè posso avere un ottimo software, ma se al cliente non piace, non è un buon software (o se non rispetta determinate normative) ogni volta che convoco clienti/stakeholder per la validazione, è un processo costoso, quindi è importante arrivare alla validazione con un sw funzionante e che rispetti le aspettative.

per questo motivo abbiamo il **testing**:

a un sw funzionante forniamo dei dati di prova (realistici) e si verifica che il sw non tui e che i risultati siano conformi alle attese.



nota: al prof non piace la freccia che va da test dei componenti al test del cliente

Test dei componenti

il componente può essere una funzione, una classe, un microservizio, ecc.

dobbiamo iniziare con il test del singolo componente (il fatto che tutti i componenti singolarmente funzionino non significa che il sistema complessivo funzioni)

un esempio di un sistema che funziona anche se un singolo componente non funziona:

una fotocamera che deve determinare la traiettoria di un oggetto in movimento, se un singolo è alterato (magari per una goccia di pioggia sul sensore), il modello del sensore fornisce un dato, ma il sistema, basandosi sul contesto e gli altri sensori/dati è in grado di correggere l'errore e fornire comunque un risultato corretto.

Test del sistema

una volta testati tutti i singoli componenti, si passa al test del sistema completo.

come accennato prima, se alcuni componenti funzionano bene, ma non sono stati scritti con



interfacce compatibili (o altri problemi) il sistema nel complesso non funziona.
bisogna quindi fare del debugging e capire dove si trova il problema

Test del cliente

il sistema viene testato con i dati reali di un cliente in condizioni realistiche.
questo test può rilevare problemi con i requisiti
ci sono dei margini di tolleranza

Test iterativo

Alcuni errori nei componenti del programma possono apparire durante il test del sistema o il cliente può stimolare il sistema in modi non previsti dai dati simulati



Evoluzione

solitamente non bisognerebbe mai di "reinventare la ruota", ma è importante usare sistemi esistenti (e già testati esaustivamente) e modificarli per le proprie esigenze.

Con l'uso intensivo di LLMs, si tende però a riscrivere librerie già esistenti da 0, perchè la velocità con cui un LLM produce codice è paragonabile al cercare una libreria già esistente e adattarla alle proprie esigenze.

Nel corso della sua vita, il sw viene modificato continuamente:

- manutenzione correttiva
- migliorie (efficienza o funzionalità)
- addattamento a nuovi mutamenti del contesto



Modelli di processi software

DEFINIZIONE modello del ciclo di vita del software (CVS):

Un modello del ciclo di vita del software è una caratterizzazione descrittiva o prescrittiva di come un sistema software viene o dovrebbe essere sviluppato

- (W. Scacchi in Encyclopedia of Software Engineering)

DEFINIZIONE STANDARD ISO/IEC/IEEE 12207:2026:

Software Life Cycle:

evolution of a system, product, service, project or other human-made entity from conception through retirement

Life Cycle Model:

framework of processes and activities concerned with the life cycle which can be organized into stages, acting as a common reference for communication and understanding

nota: viene ancora definito **human-made entity** perchè anche se un LLM ha scritto il codice o ha contribuito nella fase di progettazione, è comunque un essere umano a dover validare il software e a doverlo rilasciare, quindi il software è sempre un prodotto umano.

Non esiste un processo universale.

Il modello mi astrae il processo di ciclo di vita reale di un sw

nel '96 la definizione era molto più complessa:

non spiegava neanche effettivamente a cosa servisse il modello ed aveva più righe di quella attuale

Ogni processo ha diverse possibili descrizioni:

- **Architetturale** – descrive la sequenza di attività senza fornire dettagli sulle specifiche attività (utilizzato in questo corso);
- **Data-flow** – evidenzia le trasformazioni dei dati operate dalle attività del processo;
- **Role/action** – I ruoli delle persone coinvolte nel processo e le relative responsabilità

perchè si parla di ruoli e non persone?

una persona può avere più ruoli, o un singolo ruolo può essere ricoperto da più persone.

la **descrizione** del processo sf include:

- Attività
- prodotti



- ruoli
- pre- e post-condizioni

Quale modello devo scegliere?

- **plan driven** (tutto deve essere pianificato in anticipo, e il processo deve seguire il piano)
- **agili** (non fortemente guidati da un piano, ma si adattano velocemente ai cambiamenti)

questi sono classi di modelli e bisogna sapere quali applicare in base al contesto, talvolta è anche solo preferenza personale.

nota: nel PW3 da pagina 13 ci sono un sacco di domande di teoria che spesso chiede all'esame